

Practical No : 02
Name : Kiran Dadarao Janjal
Roll No : 13216
Batch : B1

Program :

```
import java.util.*;
import java.util.regex.*;

public class MacroProcessorInMemory {
    private Map<String, Integer> MNT = new LinkedHashMap<>();

    private List<String> MDT = new ArrayList<>();

    private Map<String, List<String>> macroALAs = new HashMap<>();

    private List<String> intermediate = new ArrayList<>();

    private List<String> finalOutput = new ArrayList<>();

    public static void main(String[] args) {
        MacroProcessorInMemory mp = new MacroProcessorInMemory();
        mp.run();
    }

    public void run() {
        String[] input = {
            "START 100",
            "",
            "MACRO",
            "INCR &ARG1,&ARG2",
            "LOAD A, &ARG1",
            "ADD A, &ARG2",
            "STORE A, RESULT",
            "MEND",
            "",
            "MOV R1, R2",
            "INCR 5, 10",
            "END"
        };

        pass1(Arrays.asList(input));
        pass2();

        System.out.println("=== MNT (Macro Name Table) ===");
        for (Map.Entry<String, Integer> e : MNT.entrySet()) {
            System.out.println(e.getKey() + " -> MDT index " + e.getValue()
                + " | ALA: " + macroALAs.get(e.getKey()));
        }
    }
}
```

```

        System.out.println("\n=== MDT (Macro Definition Table) ===");
        for (int i = 0; i < MDT.size(); i++) {
            System.out.println(i + " : " + MDT.get(i));
        }

        System.out.println("\n=== Intermediate Code (Pass-I Output) ===");
        for (String line : intermediate) {
            System.out.println(line);
        }

        System.out.println("\n=== Final Expanded Code (Pass-II Output) ===");
        for (String line : finalOutput) {
            System.out.println(line);
        }
    }

    private void pass1(List<String> inputLines) {
        boolean inMacro = false;
        String currentMacro = null;
        List<String> currentParams = null;

        for (int i = 0; i < inputLines.size(); i++) {
            String raw = inputLines.get(i);
            String line = raw.trim();

            if (line.equalsIgnoreCase("MACRO")) {
                inMacro = true;
                i++;
                String header = inputLines.get(i).trim();
                String[] parts = header.split("\\s+", 2);
                currentMacro = parts[0];
                String params = (parts.length > 1) ? parts[1] : "";
                currentParams = parseFormalParams(params);

                MNT.put(currentMacro, MDT.size());
                macroALAs.put(currentMacro, currentParams);
                MDT.add(";HEADER " + header);
                continue;
            }

            if (inMacro) {
                if (line.equalsIgnoreCase("MEND")) {
                    MDT.add("MEND");
                    inMacro = false;
                } else {
                    String mdtLine = line;
                    for (int p = 0; p < currentParams.size(); p++) {
                        String formal = currentParams.get(p);
                        mdtLine = mdtLine.replaceAll("&" + Pattern.quote(formal),
                            "(P," + (p + 1) + ")");
                    }
                }
            }
        }
    }

```

```

        }
        MDT.add(mdtLine);
    }
} else {
    intermediate.add(raw);
}
}
}

private void pass2() {
    for (String raw : intermediate) {
        String line = raw.trim();
        if (line.isEmpty()) {
            finalOutput.add("");
            continue;
        }

        String[] parts = line.split("\\s+", 2);
        String first = parts[0];

        if (MNT.containsKey(first)) {
            int mdtIndex = MNT.get(first);
            List<String> formals = macroALAs.get(first);
            List<String> actuals = (parts.length > 1)
                ? parseActualParams(parts[1])
                : new ArrayList<>();

            Map<Integer, String> ALA = new HashMap<>();
            for (int i = 0; i < formals.size(); i++) {
                ALA.put(i + 1, (i < actuals.size()) ? actuals.get(i) : "");
            }

            for (int j = mdtIndex; j < MDT.size(); j++) {
                String mdtLine = MDT.get(j);
                if (mdtLine.equals("MEND"))
                    break;
                if (mdtLine.startsWith(";HEADER"))
                    continue;

                String expanded = mdtLine;
                for (Map.Entry<Integer, String> e : ALA.entrySet()) {
                    expanded = expanded.replaceAll("\\\\(P," + e.getKey() +
"\\)",
                        e.getValue());
                }
                finalOutput.add(expanded);
            }
        } else {
            finalOutput.add(raw);
        }
    }
}

```

```

    }
}

private List<String> parseFormalParams(String part) {
    List<String> res = new ArrayList<>();
    if (part == null || part.isEmpty())
        return res;
    for (String p : part.split(",")) {
        p = p.trim();
        if (p.startsWith("&"))
            p = p.substring(1);
        if (!p.isEmpty())
            res.add(p);
    }
    return res;
}

private List<String> parseActualParams(String part) {
    List<String> res = new ArrayList<>();
    if (part == null || part.isEmpty())
        return res;
    for (String p : part.split(",")) {
        res.add(p.trim());
    }
    return res;
}
}

```

OUTPUT :=== MNT (Macro Name Table) ===
 INCR -> MDT index 0 | ALA: [ARG1, ARG2]

=== MDT (Macro Definition Table) ===
 0 : ;HEADER INCR &ARG1,&ARG2
 1 : LOAD A, (P,1)
 2 : ADD A, (P,2)
 3 : STORE A, RESULT
 4 : MEND

=== Intermediate Code (Pass-I Output) ===
 START 100

```

MOV R1, R2
INCR 5, 10
END

```

=== Final Expanded Code (Pass-II Output) ===
 START 100

```
MOV R1, R2  
LOAD A, 5  
ADD A, 10  
STORE A, RESULT  
END
```